

Trabajo Práctico 2 — Catan

[TB025] Paradigmas de Programación
Segundo cuatrimestre de 2025

Alumno	Padrón	Email
Maurizio Giannantonio	108119	mgiannantonio@fi.uba.ar
Nicolas Martin Guerrero	112514	nguerrero@fi.uba.ar
Tomas Echeverria	111283	techeverria@fi.uba.ar
Ariel Cruz	110981	accruz@fi.uba.ar
Enzo Ponferrada	101184	enzoponf3@gmail.com

Índice

1. Introducción	2
2. Supuestos de diseño	2
3. Modelo de dominio	2
4. Diagramas de clase	3
5. Detalles de implementación	9
5.1. Gestión del ciclo de vida de las Cartas de Desarrollo	9
5.2. Carta de Construcción de Carreteras	9
5.3. ManagerTurno como Fachada del Modelo	10
5.4. Patrón Command en la implementación de Cartas de Desarrollo	11
5.5. Patrón Factory en la creación del Tablero	12
6. Excepciones	12
7. Diagrama de paquetes	13
8. Diagramas de secuencia	13

1. Introducción

El presente informe reúne la documentación correspondiente a la solución del segundo trabajo práctico de la materia Paradigmas de Programación. El objetivo del trabajo es desarrollar una adaptación del juego Catan en Java aplicando los conceptos del paradigma de la orientación a objetos, SOLID y patrones de diseño vistos hasta ahora en el curso.

2. Supuestos de diseño

Para el desarrollo del juego se adoptaron los siguientes supuestos:

- Al comenzar su turno, el jugador debe lanzar obligatoriamente los dados. Solo después de esa tirada se habilita la realización de acciones (comercio, construcción, compra de cartas, etc.).
- La cantidad máxima de jugadores admitida en la implementación actual es de cuatro (4). No obstante, el diseño del modelo permite escalar a una mayor cantidad de jugadores con cambios acotados.
- Se implementa el funcionamiento de las cartas especiales *Camino más largo* y *Gran caballería*. El jugador con la mayor cantidad de caminos continuos obtiene el bono de *Camino más largo*, y el jugador con la mayor cantidad de cartas de caballero jugadas obtiene el bono de *Gran caballería*.
- Se considera que las cartas de desarrollo solo pueden ser utilizadas a partir del turno siguiente a aquel en el que fueron adquiridas.
- Las reglas de intercambio entre jugadores son libres: los jugadores pueden negociar e intercambiar cualquier cantidad y combinación de recursos, siempre que cada uno disponga efectivamente de los recursos que ofrece.
- La compra de cartas de desarrollo, caminos, poblados y ciudades solo se habilita cuando el jugador dispone de los recursos necesarios para adquirir al menos una unidad de ese elemento.
- Se asume un tablero con una cantidad fija de puertos genéricos y puertos específicos, definidos al momento de la inicialización de la partida.
- Se asume la generación de un tablero con 19 terrenos, organizados de manera aleatoria, respetando las cantidades máximas de cada tipo de terreno según las reglas del juego base.
- Se asume que el banco dispone de una cantidad inicial limitada de recursos: veinte (20) unidades de cada tipo de recurso.
- Existe una cantidad limitada de cada tipo de carta (por ejemplo, cartas de desarrollo y cartas especiales), acorde a las restricciones del juego original.

3. Modelo de dominio

Durante el análisis de las reglas del juego, identificamos la necesidad de separar claramente dos partes del dominio del modelo: (1) La estructura física del tablero y sus elementos. (2) La dinámica de turnos y el flujo de acciones del jugador.

A partir de esta separación conceptual, definimos los dos componentes centrales del modelo:

1. Tablero El **Tablero** representa la parte “física” del juego. Modela los elementos estáticos y topológicos: hexágonos de Terreno, vértices, lados, puertos y construcciones. Su responsabilidad principal es mantener la estructura espacial, verificar las restricciones de colocación (por ejemplo, la regla de distancia, adyacencias o bloqueos) y administrar cómo interactúan los objetos que en

el juego físico son piezas. Este componente encapsula toda la lógica que depende de la colocación y la disposición del tablero, asegurando que el resto del sistema no necesite conocer detalles de representación ni coordenadas.

2. ManagerTurno

El **ManagerTurno** administra los aspectos dinámicos del juego. Coordina a los **Jugadores**, controla el orden de turnos inicial y normal, gestiona el intercambio con el banco, el uso de cartas de desarrollo, el progreso de los puntos de victoria y la evaluación de bonificaciones especiales como la Gran Caballería o la Gran Ruta Comercial. Este componente funciona como orquestador del flujo del juego, delegando al **Tablero** las operaciones que dependen de la estructura física y manteniendo en sí mismo las reglas que dependen del estado de la partida.

En conjunto, estos dos componentes principales nos permiten separar las responsabilidades de manera clara, manteniendo un diseño modular, extensible y consistente con los principios SOLID.

El modelo resultante cumple las reglas del Catan y nos facilita la incorporación posterior de la vista y los controladores, necesarios para la versión jugable del proyecto.

4. Diagramas de clase

A continuación se adjuntan los diagramas que muestran las relaciones entre los objetos siendo Catan el punto de entrada.

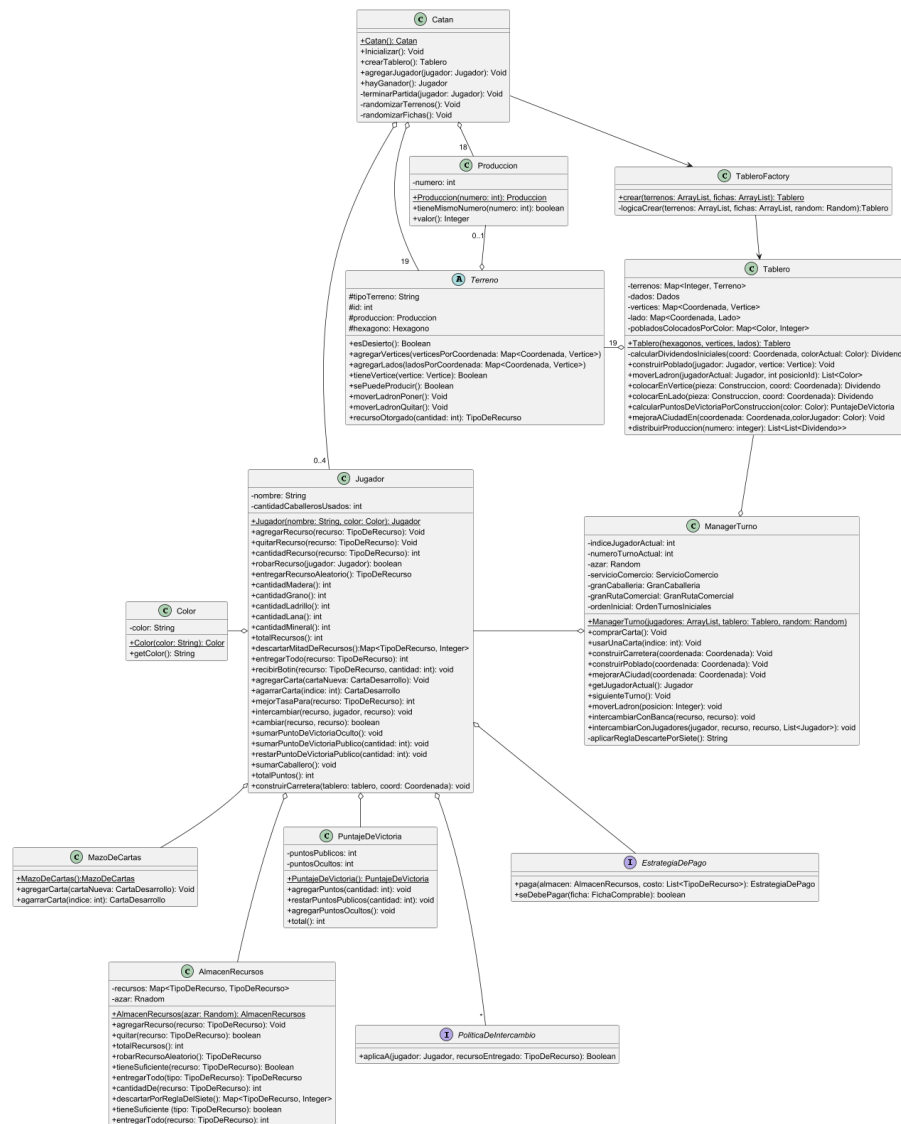


Figura 1: Principales clases del Catan.

Catan utiliza al TableroFactory para crear un tablero para iniciar el juego. ManagerTurno es el encargado del manejo de la dinámica del juego, es quién sabe de quién es el turno actual.

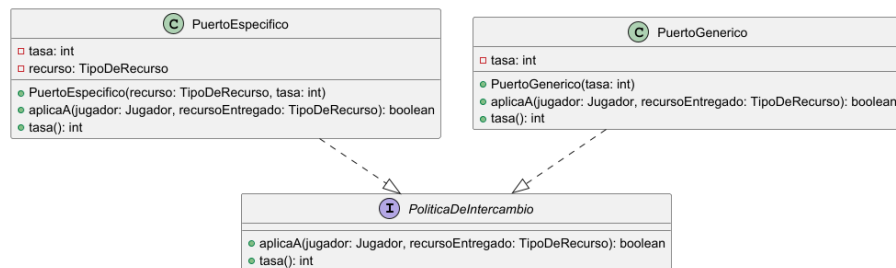


Figura 2: Política de Intercambio.

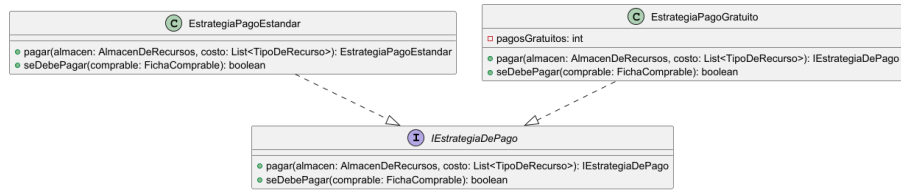


Figura 3: Estrategia de pago.

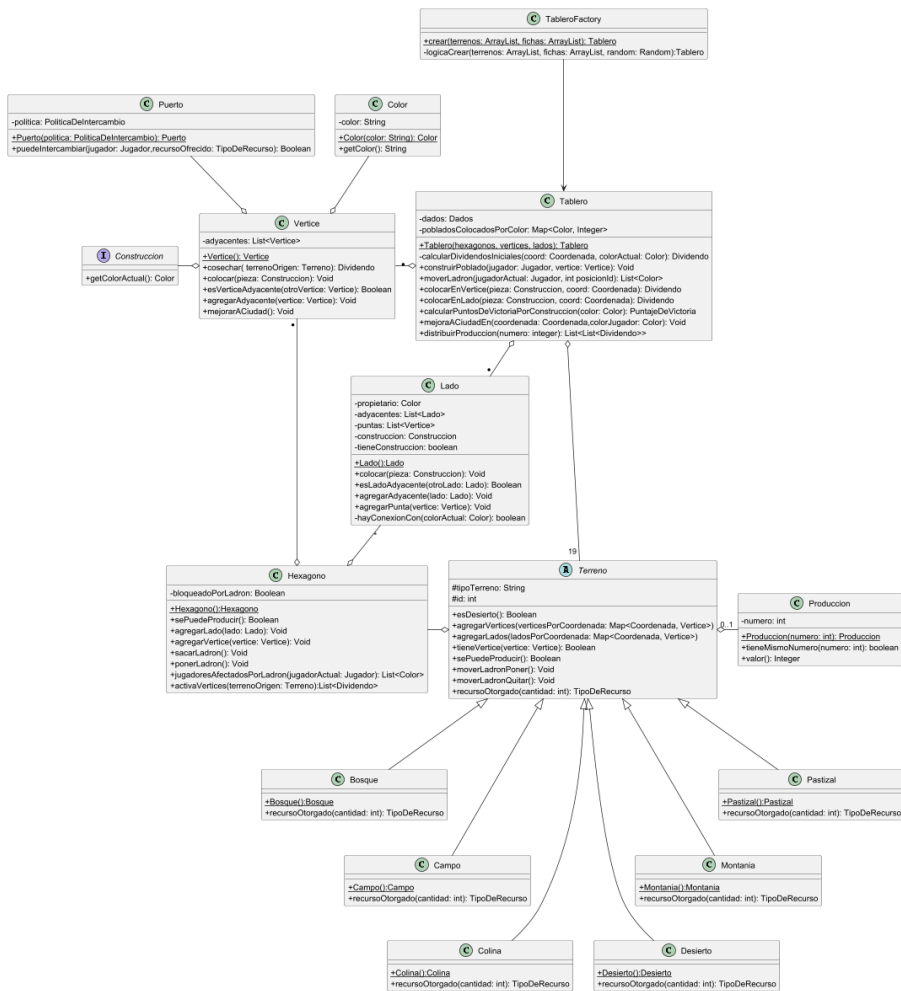


Figura 4: Relaciones del Tablero.

El Tablero conoce sus Vértices y Lados necesarios para la construcción de piezas.

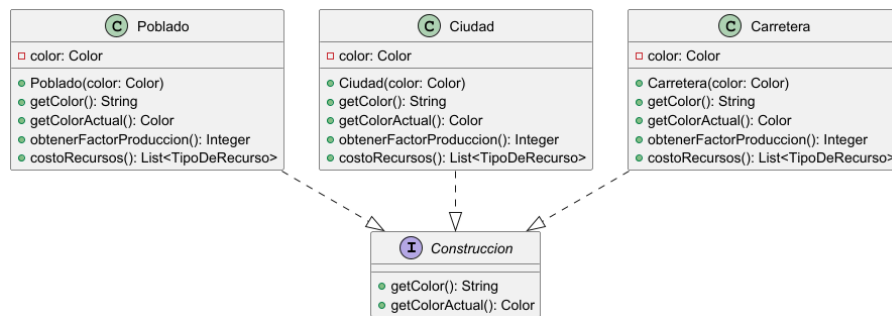


Figura 5: Construccion.

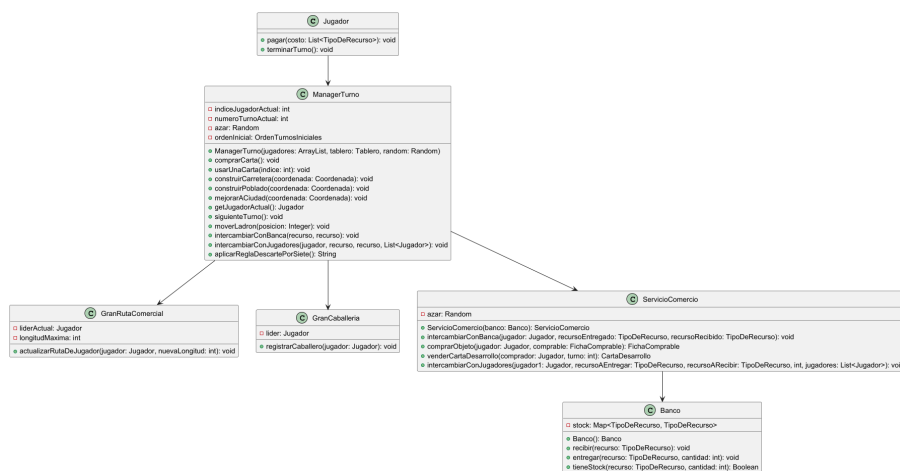


Figura 6: Relaciones del ManagerTurno.

Al manejar la dinámica del juego ManagerTurno conoce múltiples clases en las que luego delega comportamiento.



Figura 7: Relaciones del AlmacenRecursos.

Jugador delega el manejo de sus recursos en AlmacenRecursos.

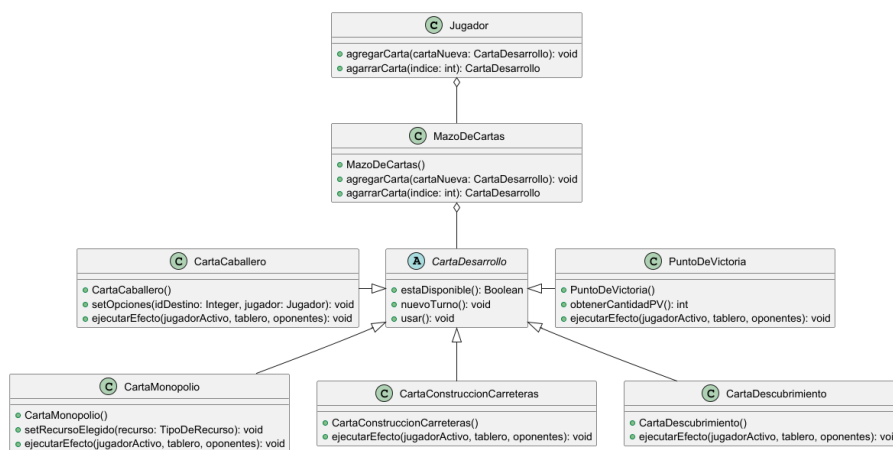


Figura 8: Relaciones del MazoDeCartas.

Jugador delega el manejo de sus cartas en MazoDeCartas.

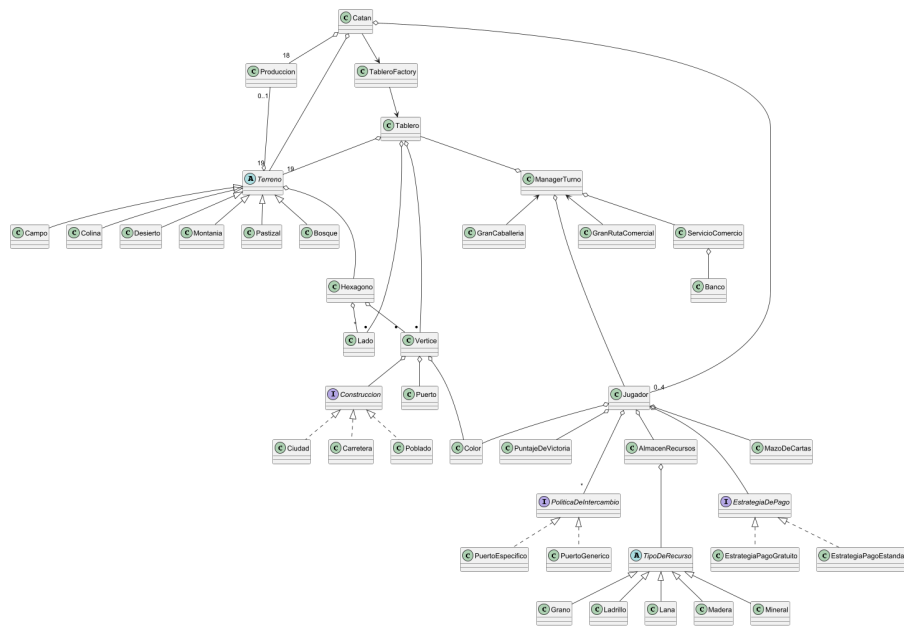


Figura 9: Diagrama esquelético de todas las clases.

5. Detalles de implementación

En el desarrollo del modelo, tal como se comentó en clase, muchas de las decisiones de diseño no fueron impuestas, sino que surgieron de la propia dinámica del problema. A continuación se detallan las más relevantes:

5.1. Gestión del ciclo de vida de las Cartas de Desarrollo

Uno de los aspectos más importantes del diseño fue la necesidad de modelar el ciclo de vida de las cartas de desarrollo. El juego impone que una carta recién adquirida no pueda usarse en ese mismo turno, que luego pase a estar disponible y que, finalmente, una vez utilizada, no pueda volver a emplearse. Para evitar condicionales repetidos y representar estas transiciones de manera natural, se aplicó el patrón *State*.

Este mecanismo se materializa mediante un conjunto de objetos que representan los distintos estados de la carta. Cada uno encapsula su propio comportamiento frente a operaciones como consultar disponibilidad, validar su uso o actualizarse al iniciar un nuevo turno. La delegación de responsabilidades en estos objetos permitió mantener el código claro, expresivo y libre de bifurcaciones innecesarias.

Las clases que llevan a la práctica este patrón son `EstadoCartaRecienComprada`, `EstadoCartaDisponible` y `EstadoCartaUsada`. Los métodos característicos se muestran a continuación:

| métodos relevantes del patrón State |

```
EstadoCartaRecienComprada >> actualizarEstado() {  
    return new EstadoCartaDisponible();  
}
```

```
EstadoCartaDisponible >> actualizarEstado() {  
    return new EstadoCartaUsada();  
}
```

```
EstadoCartaUsada >> actualizarEstado() {  
    return this;  
}
```

5.2. Carta de Construcción de Carreteras

Otro caso interesante de documentar fue la carta destinada a la construcción gratuita de dos carreteras. Aquí se evidencia nuevamente la importancia de delegar responsabilidades en lugar de concentrar la lógica en un único punto. La carta coordina la acción, pero la construcción efectiva es responsabilidad del jugador y del tablero, siguiendo así el diseño orientado a objetos.

Para modelar la gratuidad temporal de la construcción, se aplicó el patrón *Strategy*. En este caso, el jugador mantiene una estrategia de pago que puede variar durante la ejecución del turno. La carta establece una estrategia alternativa que habilita hasta dos pagos gratuitos y, una vez agotados, retorna automáticamente a la estrategia estándar. Este comportamiento se expresa a través de la interfaz de pago, que permite intercambiar dinámicamente la lógica asociada al costo de una construcción.

| métodos relevantes del patrón Strategy |

```
EstrategiaPagoEstandar >> pagar(almacen, costo) {  
    verificar recursos...  
    descontar recursos...  
    return this;  
}
```

```
EstrategiaPagoGratisito >> pagar(almacen, costo) {
    pagosGratisitos--;
    if (pagosGratisitos <= 0) {
        return new EstrategiaPagoEstandar();
    }
    return this;
}

EstrategiaPagoGratisito >> seDebePagar(comprable) {
    return !(pagosGratisitos > 0 && comprable es Carretera);
}
```

Finalmente, la carta sólo puede ejecutarse si cuenta con las dos coordenadas previamente establecidas. En caso contrario, se reporta una situación inválida, evitando que la acción avance en un estado incompleto y preservando la integridad de las reglas que regulan este tipo de construcción.

5.3. ManagerTurno como Fachada del Modelo

De la clase `ManagerTurno` emergió así una fachada que unifica el acceso a subsistemas diversos tablero, comercio, jugadores, reglas de producción y fases iniciales y expone una interfaz coherente para la vista o capa superior. De esta forma, el modelo evitó la dispersión de responsabilidades y cada componente pudo concentrarse en su lógica específica sin conocer detalles ajenos.

Tal como ocurre en el patrón *Facade*, esta clase ofrece un conjunto de operaciones de alto nivel que simplifican la interacción con estructuras internas más complejas. Acciones como comprar cartas, construir, mover el ladrón o avanzar el turno se expresan de forma directa, mientras que la lógica subyacente queda encapsulada en el interior del modelo. Esto nos permitió aplicar una arquitectura más legible y reducir el acoplamiento entre módulos.

Un ejemplo claro de este diseño puede observarse en los métodos que integran varias operaciones internas en una única acción visible para la interfaz. A continuación se incluyen algunos de los métodos característicos de esta fachada:

| métodos relevantes del patrón Facade |

```
ManagerTurno >> comprarCarta() {
    carta = servicioComercio.venderCartaDesarrollo(...)
    jugadorActual.agregarCarta(carta)
    ...
}

ManagerTurno >> construirCarretera(coordenada) {
    objeto = servicioComercio.comprarObjeto(...)
    tablero.colocarEnLado(objeto, coordenada)
    granRutaComercial.actualizarRuta(...)
}

ManagerTurno >> moverLadron(posicion) {
    victimas = tablero.moverLadron(...)
    jugadorActual.robarRecurso(victimaAlAzar)
}

ManagerTurno >> siguienteTurno() {
    jugadorActual.terminarTurno()
    contarPuntos()
}
```

```

        actualizarIndices()
    }

```

De esta forma, `ManagerTurno` se consolidó como la puerta de entrada al modelo.

5.4. Patrón Command en la implementación de Cartas de Desarrollo

Otro caso relevante fue la evolución del manejo de las cartas de desarrollo. Pasaron de ser simples portadoras de datos a objetos capaces de ejecutar su propio comportamiento. Esta transición nos permitió respetar principios de extensibilidad y desacoplamiento, evitando que el `ManagerTurno` acumulara lógica condicional o dependencias innecesarias.

En este contexto, el patrón *Command* surgió de manera natural. La clase abstracta `CartaDesarrollo` fija el contrato mediante `ejecutarEfecto`, lo que permite tratar a todas las cartas por igual sin conocer sus detalles. Cada subclase concreta expresa su acción específica: el Caballero mueve al ladrón, Monopolio redistribuye recursos, Descubrimiento otorga bonificaciones y la carta de Construcción gestiona el pago gratuito.

De esta forma, el `ManagerTurno` actúa como invocador. Su responsabilidad se reduce a disparar la acción correspondiente sin necesidad de verificar tipos ni mantener condicionales. El comportamiento queda encapsulado en cada comando, mientras que los receptores tablero, jugadores o servicio de comercio ejecutan las operaciones necesarias.

Este enfoque mejoró la extensibilidad del sistema: nuevas cartas pueden incorporarse sin modificar el `ManagerTurno`, y la lógica de cada acción permanece organizada dentro de su propio objeto, favoreciendo el mantenimiento y evitando responsabilidades cruzadas.

A continuación se muestran algunos de los métodos característicos de esta estructura:

| Métodos relevantes del patrón Factory |

```

// Punto de entrada: crea un tablero completamente inicializado
TableroFactory >> crear(terrenos, fichas)
TableroFactory >> crear(terrenos, fichas, random)

// Orquesta todos los pasos de construcción interna
TableroFactory >> logicaCrear(terrenos, fichas, random)

// Genera posiciones axiales para los 19 hexágonos del layout
TableroFactory >> generarLayoutHexagonal()

// Reemplaza instancias duplicadas de vértices entre hexágonos adyacentes
TableroFactory >> unificarVerticesCompartidos(terrenosPorId, verticesPorCoordenada)

// Asegura que un mismo lado compartido tenga una única instancia
TableroFactory >> unificarLados(ladosPorCoordenada, verticesPorCoordenada)

// Conecta vértices y lados conforme a la estructura hexagonal
TableroFactory >> conectarPuntasYLados(ladosPorCoordenada, verticesPorCoordenada, ladosUnicos)
TableroFactory >> conectarVerticesAdyacentes(terrenosPorId, verticesPorCoordenada)

// Asigna puertos a pares de vértices, mezclando su distribución
TableroFactory >> asignarPuertos(verticesPorCoordenada, random)

```

De esta forma, el patrón *Command* contribuyó a una estructura más integrada y consistente, permitiendo que el comportamiento de cada carta se exprese de manera clara y manteniendo al modelo alineado con los objetivos de flexibilidad y claridad que guiaron el desarrollo del proyecto.

5.5. Patrón Factory en la creación del Tablero

La construcción del tablero, un proceso que requiere varios pasos coordinados y que, de no centralizarse, hubiera quedado disperso en distintas partes del código. Esta situación nos llevó a aplicar el patrón *Factory Method* en la clase **TableroFactory**.

La creación de un tablero involucra generar posiciones axiales, instanciar vértices y lados, unificar componentes compartidos, asignar producción, mezclar puertos y conectar cada elemento según las reglas geométricas del juego. Encapsular toda esta lógica en una única clase permitió ocultar los detalles internos y ofrecer una interfaz simple: **TableroFactory.crear()**. De esta forma, cualquier cliente del modelo puede obtener un tablero completamente armado sin conocer su construcción interna.

Este diseño nos permitió respetar principios de encapsulamiento y reducir el acoplamiento, ya que la inicialización del tablero dejó de ser responsabilidad de **Tablero** o del **ManagerTurno**. En cambio, se delegó en un componente especializado cuya única tarea es garantizar la creación correcta y consistente del escenario de juego.

6. Excepciones

IntercambioInvalidoException Esta excepción tiene el rol de representar situaciones en las que una operación comercial no cumple con las reglas establecidas. Se utiliza para señalar intercambios que no respetan las políticas vigentes, garantizando que cada transacción sea válida y consistente dentro del sistema.

RecursosInsuficientesException Esta excepción modela el caso en que un jugador intenta realizar una acción sin contar con los recursos necesarios. Permite detener la ejecución de manera controlada y asegurar que todas las operaciones respeten los costos definidos por las mecánicas del juego.

ReglaConstruccionException Esta excepción expresa que una acción de construcción viola alguna de las condiciones estructurales del tablero. Su función es impedir configuraciones incompatibles con las reglas, asegurando que el sistema mantenga un estado válido en todo momento.

PosInvalidaParaConstruirException Esta excepción indica que la ubicación elegida para construir no es apta para dicha acción. Su presencia evita que el modelo acepte posiciones fuera de los límites permitidos o incompatibles con la geometría del tablero.

ReglaDistanciaException Esta excepción refleja la obligatoriedad de mantener la distancia mínima entre construcciones. Se lanza cuando una nueva edificación comprometería esa separación, protegiendo la estructura del tablero y el equilibrio del juego.

VerticeOcupado Esta excepción tiene el rol de modelar el estado dinámico del tablero. Se lanza cuando un jugador intenta ocupar un vértice que ya está siendo utilizado por otra construcción, evitando inconsistencias y respetando las reglas del juego (la integridad del mapa).

CantidadIncorrectaException Esta excepción comunica que una cantidad utilizada en una operación no es válida según las reglas del juego. Su objetivo es mantener la coherencia en todas aquellas acciones que dependen de valores numéricos definidos.

MessagesFileInvalid Esta excepción representa fallas en la carga o lectura de archivos de mensajes. Permite detectar problemas de configuración sin afectar la lógica del juego, aislando correctamente los errores del entorno.

ReglaDeCompraYUsoException Esta excepción indica que el jugador intenta usar o adquirir un elemento en un momento no permitido por las reglas. Garantiza que cada acción respete la secuencia y condiciones propias del flujo del turno.

7. Diagrama de paquetes

Mostramos a continuación un diagrama de paquetes del paquete modelo. No se incluyeron en el diagrama las relaciones con las clases que no tienen un paquete en específico.

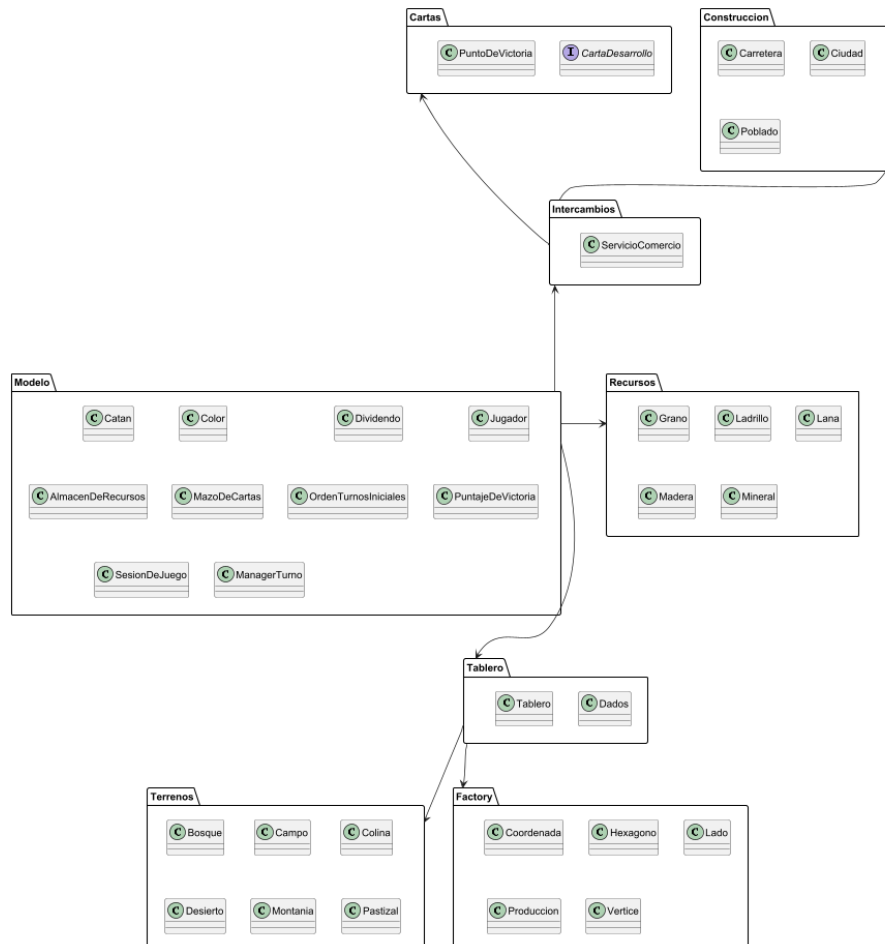
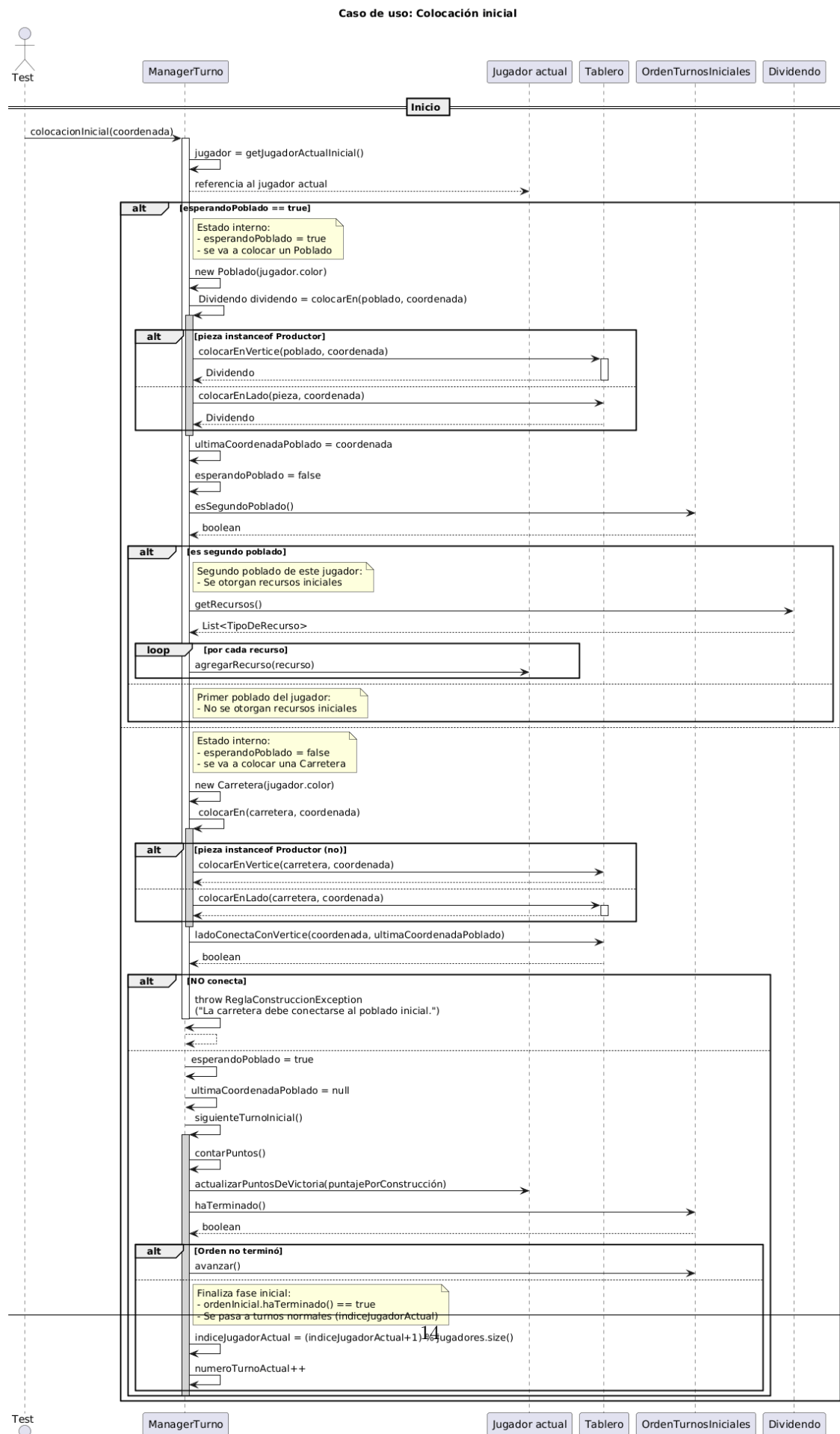


Figura 10: Diagrama de paquetes del Catan.

8. Diagramas de secuencia

Mostramos a continuación diagramas de secuencia que sobre la colocación inicial del juego, en este caso se realiza con dos jugadores para simplificar la secuencia.



La siguiente imagen muestra el correcto intercambio de recursos, donde dos jugadores intercambian recursos libremente durante el turno de uno de ellos.

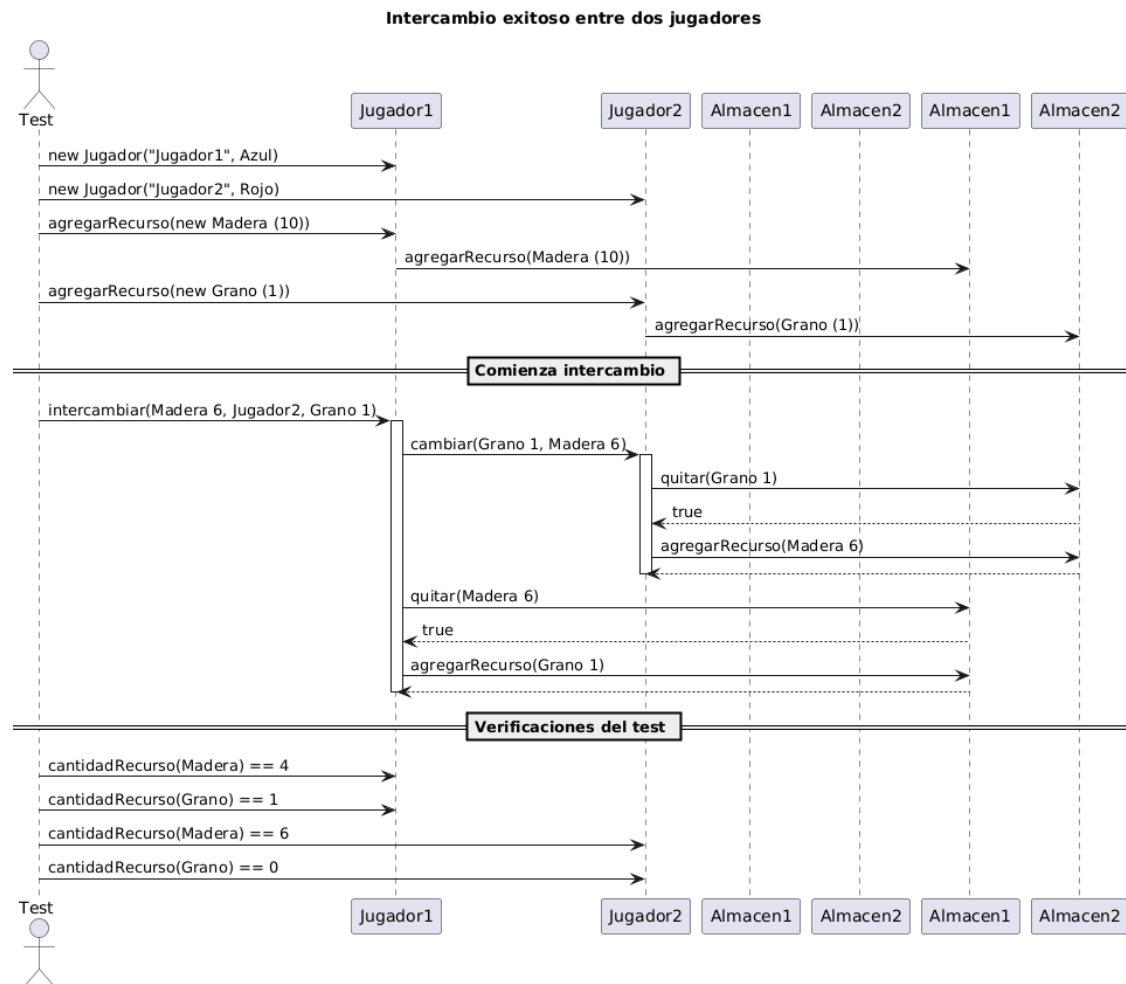


Figura 12: Intercambio correcto entre dos jugadores

El siguiente caso es el contrario al anterior donde se ve como el jugador no tiene recursos suficientes para hacer el intercambio y se lanza la excepción.

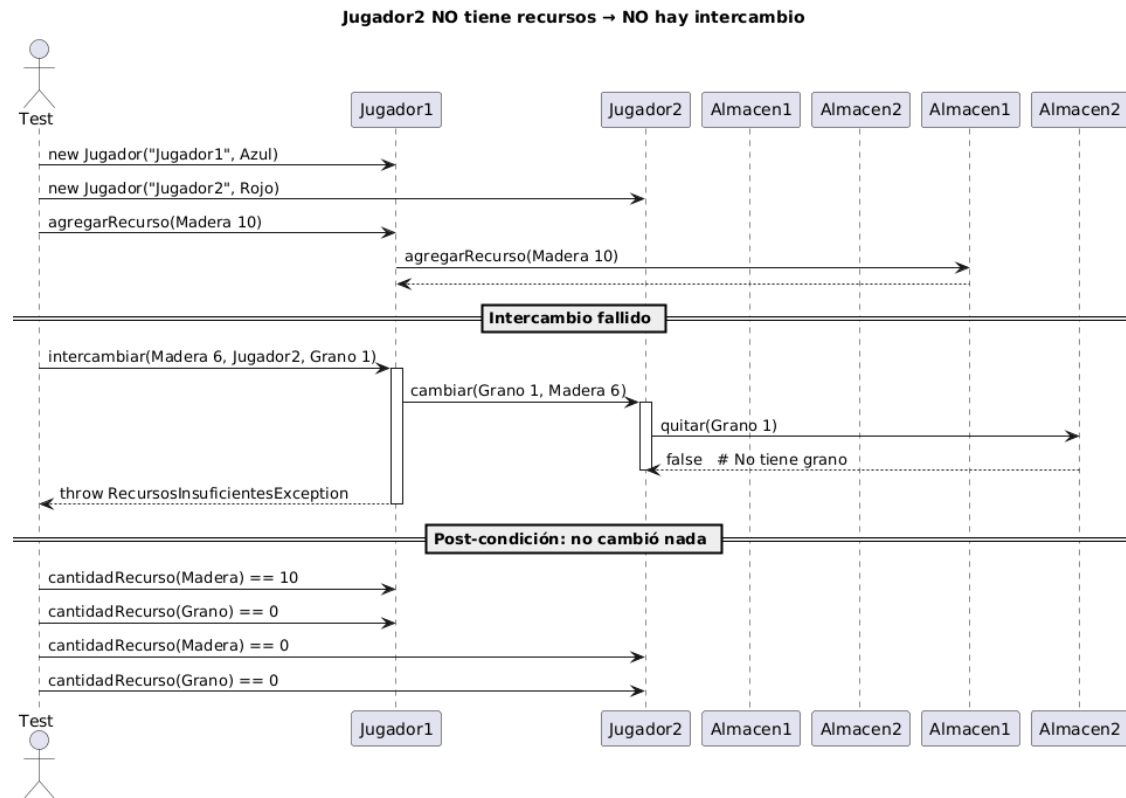


Figura 13: Intercambio incorrecto entre dos jugadores